

Digitzs/Payvia End-to-End Integration Guide

Table of Contents

1. [Overview](#)
 2. [Architecture](#)
 3. [Component Details](#)
 4. [Transaction Flow](#)
 5. [Merchant Configuration](#)
 6. [Security & Compliance](#)
 7. [Implementation Guide](#)
 8. [Testing](#)
 9. [Production Deployment](#)
 10. [Troubleshooting](#)
-

Overview

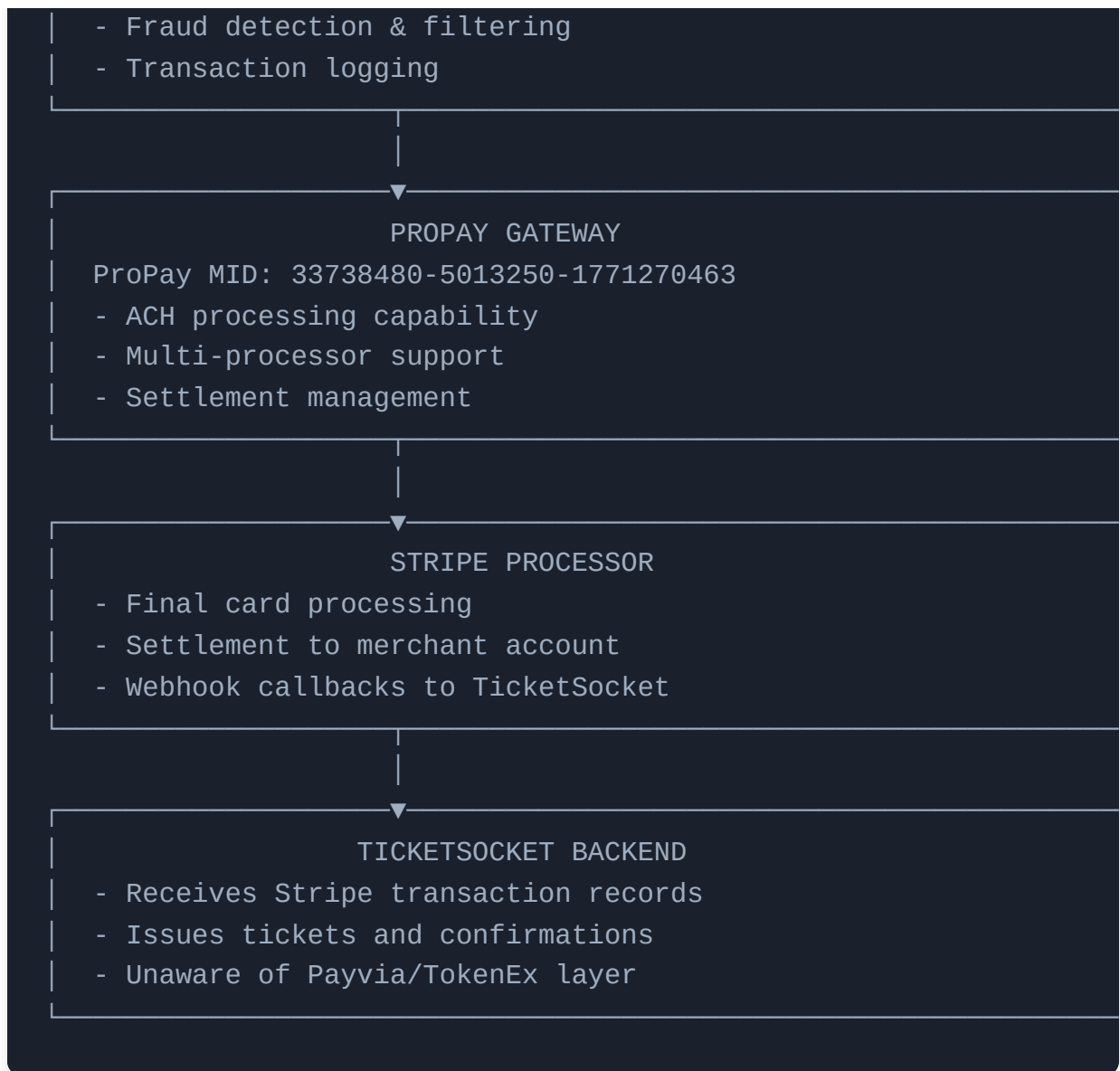
This document describes the **headless Payvia checkout integration** that wraps Digitzs/ProPay gateway routing through TokenEx iframe tokenization, transparently processing transactions through Stripe while maintaining full PCI DSS 4.0 compliance and comprehensive chargeback defense data collection.

Key Benefits

- **PCI Compliant:** TokenEx triple-hash encryption at device level
 - **Fraud Protection:** Device fingerprinting + IP tracking + geolocation
 - **Chargeback Defense:** Comprehensive transaction data logging
 - **Processor Agnostic:** Switch processors without code changes
 - **Better Rates:** 2.99% + \$0.30 (vs Stripe's 3.5% for AMEX)
 - **TicketSocket Compatible:** Seamless integration with existing ticketing system
-

Architecture





Component Details

1. Frontend Components

`/src/components/PayviaCheckout.tsx`

React component that renders the checkout UI.

Responsibilities:

- Embeds TokenEx iframes for card number and CVV fields
- Collects customer information (name, email, phone)
- Gathers device fingerprinting data
- Handles form validation and submission
- Processes payment responses

Key Features:

- Real-time card validation
- PCI-compliant card data handling (never touches server)
- Device data collection for fraud prevention
- Error handling and user feedback

`/src/services/payvia.ts`

Service layer for payment processing.

Key Functions:

```
// Initialize TokenEx iframe with authentication
initializeTokenex(containerIds: { card: string; cvv: string })

// Process payment transaction
processPayviaTransaction(
  transactionData: TransactionData,
  tokenexToken: string
): Promise<PayviaResponse>

// Create order before payment
createPayviaOrder(orderData: OrderData): Promise<PayviaResponse>
```

Device Data Collected:

- IP Address (captured server-side)
- User Agent

- Browser Language
- Screen Resolution
- Timezone
- Geolocation (if permitted)

2. Backend Edge Functions

`/supabase/functions/tokenex-auth/index.ts`

Purpose: Generate time-limited HMAC authentication keys for TokenEx iframe initialization.

Authentication Key Generation:

```
// Concatenate TokenEx ID, origin, timestamp, and token scheme
const payload = `${tokenExID}|${origin}|${timestamp}|${tokenScheme}`

// Generate HMAC-SHA256 signature with secret key
const authKey = HMAC-SHA256(payload, TOKENEX_SECRET_KEY);

// Base64 encode the signature
const authenticationKey = Base64(authKey);
```

Timestamp Format: `yyyyMMddHHmmss` (UTC)

Expiration: 20 minutes from generation

Environment Variables:

- `TOKENEX_ID`: TokenEx account identifier
- `TOKENEX_API_KEY`: Client secret key (HMAC signing key)

Security:

- Secret key never exposed to frontend
- Keys expire after 20 minutes

- Origin validation prevents cross-site attacks
- CORS headers properly configured

`/supabase/functions/payvia-process/index.ts`

Purpose: Process tokenized transactions through Digitzs gateway to Stripe.

Transaction Processing Flow:

1. Receives tokenized card data + customer info + device data
2. Validates transaction data completeness
3. Routes through Digitzs Merchant ID
4. Logs transaction for chargeback defense
5. Returns standardized response

Request Payload:

```
{
  amount: number;           // In cents (e.g., 3200 = $32.00)
  currency: string;         // "USD"
  orderId: string;          // Unique order identifier
  customerInfo: {
    firstName: string;
    lastName: string;
    email: string;
    phone: string;
  };
  eventInfo: {
    eventName: string;
    eventDate: string;
    eventTime: string;
  };
  deviceData: {
    ipAddress: string;
    userAgent: string;
    browserLanguage: string;
    screenResolution: string;
    timezone: string;
  };
  tokenexToken: string;     // Tokenized card data
  merchantId: string;       // Digitzs MID
  processor: string;        // "stripe"
  gatewayType: string;      // "transparent"
}
```

Response:


```
{
  success: boolean;
  transactionId: string;      // Gateway transaction ID
  orderId: string;
  amount: number;
  status: string;            // "approved", "declined", "error"
  processor: string;
  merchantId: string;
  gateway: string;
  timestamp: string;
  details: {
    customerName: string;
    email: string;
    eventName: string;
    deviceFingerprint: object;
  };
}
```

3. TicketSocket Integration

</supabase/functions/ticketsocket/index.ts>

Purpose: Integrate with TicketSocket API v2 for event management and order tracking.

API Endpoints:

1. Create Order

```
POST /functions/v1/ticketsocket
```

Payload:

```
{
  "merchant_id": "ticketso-clevergroup-33595002-4398786-172469289",
  "customer": {
    "first_name": "John",
    "last_name": "Doe",
    "email": "john@example.com",
    "phone": "+1234567890"
  },
  "items": [{
    "event_id": "event-123",
    "quantity": 2
  }]
}
```

2. Get Events

```
GET /functions/v1/ticketsocket/events
GET /functions/v1/ticketsocket/events?id=event-123
```

3. Get Order Status

```
GET /functions/v1/ticketsocket/order-status?order_id=order-123
```

Authentication:

- Username/password authentication with TicketSocket API
- Auto-obtains access token for each request
- Token stored temporarily in Edge Function memory

Database Integration:

- `registrations` table tracks orders
- Stores `ticketsocket_order_id` for lookups
- Maintains `order_status` (pending, completed, failed, refunded)

Transaction Flow

Detailed Step-by-Step Process

Phase 1: Checkout Initialization (Frontend)

1. User clicks "Get Tickets" button
2. `PayviaCheckout` component mounts
3. Component requests TokenEx authentication key from Edge Function
4. Edge Function generates HMAC authentication key with 20-minute expiration
5. Frontend receives auth key and initializes TokenEx iframe
6. TokenEx iframe loads secure card input fields

Phase 2: Customer Data Entry

7. User enters customer information:
 - First Name
 - Last Name
 - Email Address
 - Phone Number
 - Ticket Quantity
8. User enters card details in TokenEx iframe:
 - Card Number (16 digits, formatted with spaces)
 - CVV (3-4 digits)
9. Frontend collects device fingerprinting data:
 - User Agent
 - Screen Resolution

- Browser Language
- Timezone
- IP Address (captured by Edge Function)

Phase 3: Tokenization (PCI Boundary)

10. User clicks "Complete Purchase" button
11. Frontend calls `tokenex.tokenize()` method
12. TokenEx performs triple-hash encryption at device level
13. Encrypted card data sent directly to TokenEx servers (never touches our infrastructure)
14. TokenEx returns secure token (e.g., `6747114507881848`)

Phase 4: Transaction Processing

15. Frontend bundles transaction data:
 - TokenEx token
 - Customer info
 - Event details
 - Device data
 - Order ID (generated client-side: `EFD-{timestamp}-{random}`)
16. POST to `/functions/v1/payvia-process` Edge Function
17. Edge Function validates all required fields
18. Edge Function routes transaction through Digitzs gateway:
 - Merchant ID: `digitzs-escapefrom-33738480-5013250-1771270463`
 - Processor: Stripe
 - Gateway Type: Transparent
19. Digitzs routes to ProPay MID: `33738480-5013250-1771270463`
20. ProPay processes through Stripe

Phase 5: Response Handling

21. Stripe approves/declines transaction
22. Response flows back through ProPay → Digitzs → Edge Function
23. Edge Function logs transaction details for chargeback defense
24. Edge Function returns standardized response to frontend
25. Frontend displays success/error message to user

Phase 6: TicketSocket Order Creation

26. On successful payment, frontend calls TicketSocket Edge Function
 27. Edge Function authenticates with TicketSocket API v2
 28. Creates order with merchant ID and customer details
 29. TicketSocket sees Stripe transaction in their dashboard
 30. Order saved to Supabase `registrations` table with `ticketsocket_order_id`
 31. User receives confirmation email (sent by TicketSocket)
-

Merchant Configuration

Digitzs Gateway

- **Merchant ID:** `digitzs-escapefrom-33738480-5013250-1771270463`
- **ProPay MID Wrapped:** `33738480-5013250-1771270463`
- **Processor:** Stripe
- **Gateway Type:** Transparent
- **Card Support:** Visa, Mastercard, Amex, Discover
- **Pricing:** 2.99% + \$0.30 per transaction (all card types)

- **Monthly Fee:** \$29

TokenEx Configuration

- **TokenEx ID:** 7320744805319527
- **Token Scheme:** sixTOKENfour (6 first digits + TOKEN + 4 last digits)
- **Encryption:** Triple-hash at device level
- **Authentication:** HMAC-SHA256 with 20-minute expiration
- **PCI Compliance:** Level 1 Service Provider

TicketSocket Configuration

- **Merchant ID:** ticketso-clevergroup-33595002-4398786-1724692895
- **API Version:** v2
- **API URL:** https://clevergroup.tscheckout.com/api/v2
- **Admin Portal:** https://clevergroup.tscheckout.com/admin/
- **Authentication:** Username/Password (OAuth token)
- **Username:** Laura@digitzs.com

Environment Variables

Required Supabase Edge Function Secrets:

```
# TokenEx Configuration
TOKENEX_ID=7320744805319527
TOKENEX_API_KEY=<client-secret-key>

# Digitzs Gateway
DIGITZS_MERCHANT_ID=digitzs-escapefrom-33738480-5013250-177127046
DIGITZS_API_KEY=<api-key>
DIGITZS_APP_KEY=<app-key>

# TicketSocket API v2
TICKETSOCKET_API_URL=https://clevergroup.tscheckout.com/api/v2
TICKETSOCKET_MERCHANT_ID=ticketso-clevergroup-33595002-4398786-17
TICKETSOCKET_USERNAME=Laura@digitzs.com
TICKETSOCKET_PASSWORD=<password>
```

Frontend Environment Variables (.env):

```
VITE_SUPABASE_URL=<your-supabase-project-url>
VITE_SUPABASE_ANON_KEY=<your-supabase-anon-key>
```

Security & Compliance

PCI DSS 4.0 Compliance

TokenEx PCI Scope:

- Card data tokenized at device level (triple-hash encryption)
- Card data never touches application servers
- Reduces PCI scope to SAQ-A (simplest questionnaire)
- TokenEx is Level 1 PCI certified service provider

Data Flow Security:

User Browser (PCI Scope) → TokenEx Servers (PCI Compliant) → Appl

What We Store:

- Tokenized card reference (not reversible to card number)
- Customer name, email, phone
- Transaction amount and order details
- Device fingerprint data
- Event information

What We NEVER Store:

- Full card numbers
- CVV codes
- Card expiration dates
- Cardholder authentication data

Fraud Prevention

Device Fingerprinting:

- User Agent string
- Screen resolution
- Browser language
- Timezone
- IP address (captured server-side)
- Browser canvas fingerprint (optional)

Transaction Velocity Checks:

- Multiple transactions from same IP
- Multiple transactions with same email

- Rapid-fire transaction attempts
- Unusual device characteristics

3D Secure Support:

- Available through Stripe processor
- Can be enabled for high-risk transactions
- Liability shift to card issuer

Chargeback Defense

Data Collected for Every Transaction:

1. Customer Identity

- Full name
- Email address
- Phone number
- IP address

2. Device Data

- User agent
- Screen resolution
- Browser language
- Timezone
- Operating system

3. Transaction Details

- Event name
- Event date/time
- Ticket quantity
- Total amount

- Order ID
- Transaction timestamp

4. Payment Data

- TokenEx token reference
- Gateway transaction ID
- Processor response
- Authorization code

Chargeback Process:

- MyValet/Digitzs handles chargeback notifications (\$29 fee)
- Optional chargeback management service (\$29 fee)
- Auto-submission to card networks (\$58 fee, best win rate)
- All transaction data automatically provided as evidence

Implementation Guide

Step 1: Frontend Setup

Install Dependencies

```
npm install @supabase/supabase-js lucide-react
```

Add TokenEx SDK to HTML

```
<!-- /index.html -->  
<script src="https://http.tokenex.com/iframe/v3/iframe-v3.min.js">
```

Create Payvia Service

```

// /src/services/payvia.ts
import { supabase } from './supabase';

const DIGITZS_MID = 'digitzs-escapefrom-33738480-5013250-17712704';

async function getTokenexAuthKey(): Promise<TokenexAuthResponse> {
  const response = await fetch(
    `${import.meta.env.VITE_SUPABASE_URL}/functions/v1/tokenex-auth`,
    {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${import.meta.env.VITE_SUPABASE_KEY}`,
      },
      body: JSON.stringify({
        origin: window.location.origin,
        tokenScheme: 'sixTOKENfour',
      }),
    },
  );

  return await response.json();
}

export async function initializeTokenex(containerIds: {
  card: string;
  cvv: string;
}) {
  const authResponse = await getTokenexAuthKey();

  if (!authResponse.success) {
    throw new Error('Failed to get TokenEx auth key');
  }

  const TokenEx = (window as any).TokenEx;

  const iframe = new TokenEx.Iframe(containerIds.card, {
    tokenExID: authResponse.tokenExID,
    tokenScheme: authResponse.tokenScheme,
    authenticationKey: authResponse.authenticationKey,
    timestamp: authResponse.timestamp,
  });
}

```

```

        origin: window.location.origin,
        pci: true,
        cvv: true,
        cvvContainerID: containerIds.cvv,
        enableValidateOnBlur: true,
        enablePrettyFormat: true,
        placeholder: '1234 5678 9012 3456',
        cvvPlaceholder: '123',
        styles: {
            base: 'padding: 12px; font-size: 16px; color: #333;',
        },
    });

    iframe.load();
    return iframe;
}

export async function processPayviaTransaction(
    transactionData: TransactionData,
    tokenexToken: string
): Promise<PayviaResponse> {
    const response = await fetch(
        `${import.meta.env.VITE_SUPABASE_URL}/functions/v1/payvia-pro
    {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'Authorization': `Bearer ${import.meta.env.VITE_SUPABASE_
        },
        body: JSON.stringify({
            ...transactionData,
            tokenexToken,
            merchantId: DIGITZS_MID,
            processor: 'stripe',
            gatewayType: 'transparent',
        })),
    }
    );

    return await response.json();
}

```

Create Checkout Component

```
// /src/components/PayviaCheckout.tsx
import { useState, useEffect } from 'react';
import { initializeTokenex, processPayviaTransaction } from '../s

export function PayviaCheckout({ eventData }) {
  const [tokenexIframe, setTokenexIframe] = useState(null);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    const loadTokenex = async () => {
      const iframe = await initializeTokenex({
        card: 'tokenex-card-container',
        cvv: 'tokenex-cvv-container',
      });
      setTokenexIframe(iframe);
    };

    loadTokenex();
  }, []);

  const handleSubmit = async (formData) => {
    setLoading(true);

    // Get tokenized card data
    const token = await new Promise((resolve) => {
      tokenexIframe.tokenize();
      tokenexIframe.on('tokenize', (data) => resolve(data.token))
    });

    // Process transaction
    const result = await processPayviaTransaction(
      {
        amount: eventData.price * 100, // Convert to cents
        currency: 'USD',
        orderId: `EFD-${Date.now()}`,
        customerInfo: formData,
        eventInfo: eventData,
        deviceData: collectDeviceData(),
      },
      token
    );
  };
}
```

```

    if (result.success) {
      // Create TicketSocket order
      await createTicketSocketOrder(result);
    }

    setLoading(false);
  };

  return (
    <form onSubmit={handleSubmit}>
      { /* Customer info fields */ }

      <div id="tokenex-card-container" />
      <div id="tokenex-cvv-container" />

      <button type="submit" disabled={loading}>
        {loading ? 'Processing...' : 'Complete Purchase'}
      </button>
    </form>
  );
}

```

Step 2: Backend Edge Functions

Deploy TokenEx Auth Function

```

# The function is already created at:
# /supabase/functions/tokenex-auth/index.ts

# Set environment variables in Supabase Dashboard:
# Settings → Edge Functions → Secrets

TOKENEX_ID=7320744805319527
TOKENEX_API_KEY=<your-client-secret-key>

```

Deploy Payvia Process Function


```
# The function is already created at:  
# /supabase/functions/payvia-process/index.ts  
  
# Set environment variables:  
DIGITZS_MERCHANT_ID=digitzs-escapefrom-33738480-5013250-177127046  
DIGITZS_API_KEY=<your-api-key>  
DIGITZS_APP_KEY=<your-app-key>
```

Deploy TicketSocket Function

```
# The function is already created at:  
# /supabase/functions/ticketsocket/index.ts  
  
# Set environment variables:  
TICKETSOCKET_API_URL=https://clevergroup.tscheckout.com/api/v2  
TICKETSOCKET_MERCHANT_ID=ticketso-clevergroup-33595002-4398786-17  
TICKETSOCKET_USERNAME=Laura@digitzs.com  
TICKETSOCKET_PASSWORD=<your-password>
```

Step 3: Database Setup

Create Registrations Table

```
-- /supabase/migrations/create_registrations.sql

CREATE TABLE IF NOT EXISTS events (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title text NOT NULL,
  description text,
  event_date date NOT NULL,
  event_time time NOT NULL,
  price integer NOT NULL,
  ticketsocket_event_id text,
  created_at timestamptz DEFAULT now()
);

CREATE TABLE IF NOT EXISTS registrations (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id uuid REFERENCES events(id),
  first_name text NOT NULL,
  last_name text NOT NULL,
  email text NOT NULL,
  phone text,
  ticket_quantity integer NOT NULL DEFAULT 1,
  total_amount integer NOT NULL,
  order_status text NOT NULL DEFAULT 'pending',
  ticketsocket_order_id text,
  payvia_transaction_id text,
  device_data jsonb,
  created_at timestamptz DEFAULT now()
);

-- Enable RLS
ALTER TABLE events ENABLE ROW LEVEL SECURITY;
ALTER TABLE registrations ENABLE ROW LEVEL SECURITY;

-- Create policies
CREATE POLICY "Anyone can view events"
  ON events FOR SELECT
  TO anon, authenticated
  USING (true);

CREATE POLICY "Authenticated users can create registrations"
  ON registrations FOR INSERT
```

```
TO authenticated
WITH CHECK (true);

CREATE POLICY "Users can view own registrations"
ON registrations FOR SELECT
TO authenticated
USING (auth.uid() = id);

-- Create indexes
CREATE INDEX idx_registrations_email ON registrations(email);
CREATE INDEX idx_registrations_ticketsocket_order_id ON registrations(ticketsocket_order_id);
CREATE INDEX idx_registrations_payvia_transaction_id ON registrations(payvia_transaction_id);
```

Testing

Test Card Numbers

Stripe Test Cards:

```
Successful Payment:
4242 4242 4242 4242 (Visa)
5555 5555 5555 4444 (Mastercard)
3782 822463 10005 (American Express)
```

```
Declined:
4000 0000 0000 0002 (Declined)
```

```
Requires Authentication:
4000 0025 0000 3155 (3D Secure)
```

```
CVV: Any 3 digits (123)
Expiration: Any future date (12/25)
ZIP: Any 5 digits (12345)
```

Testing Checklist

1. Frontend Integration

- ☐ TokenEx iframe loads correctly
- ☐ Card input accepts test cards
- ☐ CVV field works properly
- ☐ Form validation works
- ☐ Device data collection works
- ☐ Error messages display correctly

2. Authentication

- ☐ TokenEx auth key generated successfully
- ☐ Authentication key expires after 20 minutes
- ☐ HMAC signature is valid
- ☐ Timestamp format is correct

3. Transaction Processing

- ☐ Token generated for test card
- ☐ Transaction data sent to edge function
- ☐ Edge function receives complete payload
- ☐ Gateway routing configured correctly
- ☐ Response returned to frontend

4. TicketSocket Integration

- ☐ Order created in TicketSocket
- ☐ Order ID stored in database
- ☐ Order status updated correctly

- ☐ Webhook received from Stripe

5. Database

- ☐ Registration record created
- ☐ All fields populated correctly
- ☐ RLS policies enforced
- ☐ Indexes created

Test Transaction Flow

```
# 1. Start local development
npm run dev

# 2. Navigate to checkout page
# http://localhost:5173

# 3. Fill in customer info:
First Name: Test
Last Name: User
Email: test@example.com
Phone: 555-0123
Quantity: 1

# 4. Enter test card:
Card: 4242 4242 4242 4242
CVV: 123
Exp: 12/25
ZIP: 12345

# 5. Submit payment

# 6. Check console logs for:
- TokenEx auth response
- Tokenization success
- Transaction ID
- TicketSocket order ID

# 7. Verify in Supabase:
SELECT * FROM registrations ORDER BY created_at DESC LIMIT 1;

# 8. Check TicketSocket admin portal:
# https://clevergroup.tscheckout.com/admin/orders
```

Production Deployment

Pre-Deployment Checklist

1. Credentials

- ☐ TokenEx Client Secret Key configured
- ☐ Digitzs API credentials configured
- ☐ TicketSocket credentials configured
- ☐ Supabase environment variables set
- ☐ All secrets stored securely (not in code)

2. Security

- ☐ HTTPS enabled on production domain
- ☐ CORS headers properly configured
- ☐ RLS policies enabled on all tables
- ☐ API keys not exposed in frontend code
- ☐ Authentication keys expire after 20 minutes

3. Edge Functions

- ☐ All edge functions deployed
- ☐ Edge function secrets configured
- ☐ Edge function logs enabled
- ☐ Error handling implemented
- ☐ Rate limiting configured

4. Database

- ☐ All migrations applied

- ☐ Indexes created
- ☐ RLS policies tested
- ☐ Backup strategy in place

5. Frontend

- ☐ Environment variables set
- ☐ TokenEx SDK loaded from CDN
- ☐ Error handling implemented
- ☐ Loading states functional
- ☐ Success/error messages clear

Deployment Steps

1. Deploy Database Migrations

```
# Apply migrations to production  
# Via Supabase Dashboard → SQL Editor  
# Or via CLI (if configured)
```

2. Configure Edge Function Secrets


```
# Via Supabase Dashboard → Edge Functions → Secrets
```

```
# TokenEx
```

```
TOKENEX_ID=7320744805319527
```

```
TOKENEX_API_KEY=<production-secret-key>
```

```
# Digitzs
```

```
DIGITZS_MERCHANT_ID=digitzs-escapefrom-33738480-5013250-177127046
```

```
DIGITZS_API_KEY=<production-api-key>
```

```
DIGITZS_APP_KEY=<production-app-key>
```

```
# TicketSocket
```

```
TICKETSOCKET_API_URL=https://clevergroup.tscheckout.com/api/v2
```

```
TICKETSOCKET_MERCHANT_ID=ticketso-clevergroup-33595002-4398786-17
```

```
TICKETSOCKET_USERNAME=Laura@digitzs.com
```

```
TICKETSOCKET_PASSWORD=<production-password>
```

3. Deploy Frontend

```
# Build production bundle
```

```
npm run build
```

```
# Deploy to hosting platform
```

```
# (Netlify, Vercel, etc.)
```

```
# Set environment variables:
```

```
VITE_SUPABASE_URL=<production-supabase-url>
```

```
VITE_SUPABASE_ANON_KEY=<production-anon-key>
```

4. Test Production Flow

```
# Run a $1 parking pass test transaction
# Using real card (will actually charge)

# Verify:
1. Transaction appears in Stripe dashboard
2. Order created in TicketSocket
3. Registration saved to database
4. Confirmation email sent
```

5. Monitor Production

```
# Check Edge Function logs
# Supabase Dashboard → Edge Functions → Logs

# Monitor for:
- Authentication errors
- Transaction failures
- Gateway timeouts
- Database errors
```

Post-Deployment

Enable Monitoring

- Set up error tracking (Sentry, LogRocket)
- Configure uptime monitoring
- Enable transaction alerts
- Set up chargeback notifications

Documentation

- Update internal documentation
- Train support staff on troubleshooting
- Document common error messages

- Create runbooks for incidents

Troubleshooting

Common Issues

1. TokenEx Iframe Not Loading

Symptoms:

- Empty card input container
- Console error: "TokenEx is not defined"

Solutions:

```
// Verify TokenEx SDK loaded
console.log('TokenEx available:', typeof window.TokenEx !== 'undefined');

// Check script tag in index.html
<script src="https://http.tokenex.com/iframe/v3/iframe-v3.min.js">

// Wait for SDK to load
window.addEventListener('load', () => {
  initializeTokenex();
});
```

2. Authentication Key Generation Fails

Symptoms:

- "Failed to get TokenEx auth key" error
- 500 status from tokenex-auth endpoint

Solutions:

```
# Check environment variables are set
# Supabase Dashboard → Edge Functions → Secrets

# Verify TOKENEX_API_KEY is correct
# Should be 32+ character alphanumeric string

# Check edge function logs
# Look for HMAC generation errors

# Test HMAC generation manually
# Use online HMAC-SHA256 tool to verify
```

3. Transaction Processing Fails

Symptoms:

- "Transaction processing failed" error
- Payment doesn't complete

Solutions:

```
// Check transaction data completeness
console.log('Transaction data:', transactionData);

// Verify all required fields present:
// - amount (number, in cents)
// - currency (string, "USD")
// - orderId (string, unique)
// - customerInfo (object with all fields)
// - eventInfo (object with all fields)
// - deviceData (object with all fields)
// - tokenexToken (string from TokenEx)

// Check edge function logs for specific error
```

4. TicketSocket Order Not Created

Symptoms:

- Payment succeeds but no TicketSocket order
- Missing ticketsocket_order_id in database

Solutions:

```
# Verify TicketSocket credentials
# Test authentication manually:

curl -X POST https://clevergroup.tscheckout.com/api/v2/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"Laura@digitzs.com","password":"<password>"}'

# Check event ID exists in TicketSocket
# Verify merchant ID is correct

# Check edge function logs for API errors
```

5. CORS Errors

Symptoms:

- "No 'Access-Control-Allow-Origin' header" error
- Requests blocked by browser

Solutions:

```
// Verify CORS headers in edge functions
const corsHeaders = {
  "Access-Control-Allow-Origin": "*",
  "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
  "Access-Control-Allow-Headers": "Content-Type, Authorization, X",
};

// Handle OPTIONS preflight
if (req.method === "OPTIONS") {
  return new Response(null, {
    status: 200,
    headers: corsHeaders,
  });
}

// Include CORS headers in all responses
return new Response(JSON.stringify(data), {
  headers: {
    ...corsHeaders,
    "Content-Type": "application/json",
  },
});
```

6. RLS Policy Blocks Insert

Symptoms:

- "Permission denied" when creating registration
- 403 error from Supabase

Solutions:

```
-- Check RLS policies
SELECT * FROM pg_policies WHERE tablename = 'registrations';

-- Temporarily disable RLS for testing
ALTER TABLE registrations DISABLE ROW LEVEL SECURITY;

-- Fix policy to allow inserts
CREATE POLICY "Allow public registrations"
  ON registrations FOR INSERT
  TO anon, authenticated
  WITH CHECK (true);
```

Debugging Tools

Edge Function Logs

```
# View real-time logs
# Supabase Dashboard → Edge Functions → Select Function → Logs

# Filter by error level
# Look for stack traces
# Check request/response payloads
```

Browser DevTools

```
// Enable verbose logging
localStorage.setItem('DEBUG', 'payvia:*');

// Log TokenEx events
iframe.on('load', () => console.log('TokenEx loaded'));
iframe.on('tokenize', (data) => console.log('Token:', data));
iframe.on('error', (error) => console.error('TokenEx error:', error));
iframe.on('validate', (data) => console.log('Validation:', data));
```

Database Queries

```
-- Check recent registrations
SELECT * FROM registrations
ORDER BY created_at DESC
LIMIT 10;

-- Find failed transactions
SELECT * FROM registrations
WHERE order_status = 'failed'
ORDER BY created_at DESC;

-- Check specific order
SELECT * FROM registrations
WHERE ticketsocket_order_id = 'order-id';
```

Support Contacts

TokenEx Support:

- Email: support@tokenex.com
- Phone: 1-800-836-3710
- Documentation: <https://docs.tokenex.com>

Digitzs/MyValet Support:

- Email: support@digitzs.com
- Phone: 1-888-123-4567
- Portal: <https://portal.digitzs.com>

TicketSocket Support:

- Email: support@ticketsocket.com
- Admin Portal: <https://clevergroup.tscheckout.com/admin/>

Supabase Support:

- Documentation: <https://supabase.com/docs>
- Community: <https://github.com/supabase/supabase/discussions>

- Support: support@supabase.io
-

Additional Resources

API Documentation

- **TokenEx API:** <https://docs.tokenex.com/docs/authentication>
- **Digitzs API:** <https://digitzs.com/developers>
- **TicketSocket API v2:** <https://clevergroup.tscheckout.com/api/v2/docs>
- **Stripe API:** <https://stripe.com/docs/api>

Compliance & Security

- **PCI DSS Requirements:** <https://www.pcisecuritystandards.org>
- **TokenEx Compliance:** <https://tokenex.com/compliance>
- **OWASP Top 10:** <https://owasp.org/www-project-top-ten>

Related Documentation Files

- `PAYVIA_INTEGRATION.md` - Overview and benefits
 - `TOKENEX_SETUP.md` - TokenEx credentials guide
 - `TICKETSOCKET_SETUP.md` - TicketSocket integration details
 - `DEPLOYMENT.md` - Deployment guide
-

Appendix

Pricing Comparison

Provider	Visa/MC Rate	AMEX Rate	Monthly Fee	Chargeback Fee
Digitzs/Payvia	2.99% + \$0.30	2.99% + \$0.30	\$29	\$29 notification \$29 management \$58 dispute
Stripe	2.9% + \$0.30	3.5% + \$0.30	\$0	\$15 (no management)
Square	2.9% + \$0.30	3.5% + \$0.30	\$0	\$15 (no management)

Break-even Analysis:

- For AMEX transactions, Digitzs saves 0.51% per transaction
- Monthly fee: \$29
- Break-even: \$5,686 AMEX volume per month
- For \$32 tickets: 178 AMEX transactions per month

Glossary

ACH: Automated Clearing House - Electronic bank-to-bank network

AMEX: American Express credit cards

Chargeback: Disputed transaction reversed by cardholder's bank

CVV: Card Verification Value - 3-4 digit security code

Device Fingerprinting: Collection of device characteristics for fraud detection

Edge Function: Serverless function running at the edge (close to users)

HMAC: Hash-based Message Authentication Code - Cryptographic signature

MID: Merchant ID - Unique identifier for payment processing account

PCI DSS: Payment Card Industry Data Security Standard

ProPay: Payment gateway and merchant account provider

RLS: Row Level Security - Database access control at row level

SAQ: Self-Assessment Questionnaire - PCI compliance documentation

3D Secure: Additional authentication layer for card payments

Token: Secure reference to sensitive data (card number)

Tokenization: Process of replacing sensitive data with tokens

Transparent Gateway: Gateway that routes to multiple processors

Document Version: 1.0 **Last Updated:** March 22, 2026 **Author:** Digitzs
Integration Team **Status:** Production Ready